

LOOP'S CHALLENGE COURSE

FULL CODING CHEAT SHEET - Patterns, Loops, Conditionals, and Flow Control

START -> CHECK -> DO -> STEP -> REPEAT

1. Loop's Core Memory Rule

Use this same path every time you read a basic loop. Say the steps out loud before you run the code.

```
for (let i = 0; i < 5; i++) {  
  // action  
}
```

START	let i = 0 - create the counter and choose its first value.
CHECK	i < 5 - ask a true/false question. True means keep going; false means stop.
DO	The code inside { } runs once for this iteration.
STEP	i++ changes the counter after the body runs.
REPEAT	Go back to CHECK and follow the path again.
Iteration	One complete pass through the loop body.

2. Full For-Loop Anatomy

for	Starts a controlled repetition pattern.
let i = 0	START. let creates the variable; i is the counter; = 0 stores the starting value.
i < 5	CHECK/STOP rule. Values are 0, 1, 2, 3, 4. When i becomes 5, the check is false.
i++	STEP. Add 1 after each pass. Same idea as i = i + 1.
i--	Subtract 1 after each pass. Useful for counting down.
{ }	The repeated body - the DO/action.
;	Separates START, CHECK, and STEP inside a for loop.

```
i = 0 -> 0 < 5? yes -> run  
i = 1 -> 1 < 5? yes -> run  
i = 2 -> 2 < 5? yes -> run  
i = 3 -> 3 < 5? yes -> run  
i = 4 -> 4 < 5? yes -> run  
i = 5 -> 5 < 5? no -> STOP
```

3. Counter Variables and Dynamic Output

```
for (let step = 1; step <= 3; step++) {  
  console.log("Lap " + step);  
}
```

Counter variable	Tracks loop progress. It can be named i, index, step, or another clear name.
Dynamic output	Text can stay the same while the counter value changes each iteration.
Off-by-one check	Always ask whether the stop rule should use < or <=.

4. Arrays + Index

```
let crew = ["Loop", "Zen", "Nova"];
crew[0] // "Loop"
crew[1] // "Zen"
crew.length // 3
```

Array	An ordered list held inside square brackets [].
Index	The numbered position of an item. Array positions start at 0.
.length	The number of items in the array.
Why i < crew.length?	For 3 items, valid indexes are 0, 1, 2. The condition stops before i reaches 3, avoiding an invalid index.

```
for (let i = 0; i < crew.length; i++) {
  console.log(crew[i]);
}
```

5. Conditionals + Booleans

```
if (condition) {
  // true path
} else if (anotherCondition) {
  // another true path
} else {
  // fallback path
}
```

Boolean	A value that is only true or false .
if	Ask a question. Run its block only if the condition is true.
else if	Check another condition if earlier branches were false.
else	Fallback path when no earlier condition was true.
Branch order	Conditions are checked top to bottom. The first true branch wins.

6. Comparison Operators

===	Exactly equal.
!==	Not equal.
>	Greater than.
<	Less than.
>=	Greater than or equal.
<=	Less than or equal.
= vs ===	One = stores. Three === asks. = assigns a value; === compares values.
&& / / !	AND / OR / NOT - combine or flip Boolean conditions.

```
let score = 10; // store 10
score === 10 // ask: exactly equal? true
```

7. For vs. While

Use for	When START / CHECK / STEP is known or predictable.
Use while	When repetition mainly depends on a condition and the exact number of passes may be unknown.

```
let energy = 10;
while (energy > 0) {
  console.log("Energy: " + energy);
  energy -= 3;
}
```

WHILE SAFETY: Something inside the loop must move the condition toward false. Otherwise the loop may never stop.

8. Nested Loops

```
for (let row = 1; row <= 3; row++) {
  for (let col = 1; col <= 3; col++) {
    console.log(row, col);
  }
}
```

Outer loop	Think rows or larger groups.
Inner loop	Think columns or smaller steps.
Key rule	For every one outer-loop pass, the inner loop completes all of its passes.

9. break vs. continue

break	EXIT the loop completely.
continue	SKIP the rest of this pass and keep looping.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) { continue; }
  console.log(i); // 1, 2, 4, 5
}
```

10. Common Loop Bugs / Debug Box

Runs one time too many	Check < vs <=.
Array says undefined	Check the index and .length . Did you go one position too far?
Loop never stops	Check the STEP/update. Is the condition moving toward false?
Wrong direction	Check ++ vs --.
Condition never runs	Test with values you know should produce true and false.
Unexpected path	Read conditions top to bottom and predict which branch becomes true first.

11. Vocabulary Quick Scan

Pattern	Something that repeats predictably.
Sequence	Steps in a specific order.

Automate	Let the computer repeat a task instead of doing it manually each time.
Flow	The path a program follows.
Counter variable	Changing value that tracks loop progress.
Conditional	Code that chooses a path based on true/false.
Comparison operator	A symbol that compares values.
Outer / inner loop	The outside and inside loops in a nested structure.
Infinite loop	A loop that never reaches a stopping condition.
Flow control	Tools that decide what repeats, which branch runs, and when execution exits or skips.

12. Prediction Checklist

Before pressing Run, answer these five questions:

1	Where does it START?
2	What CHECK must stay true?
3	What code will DO/run?
4	What changes in the STEP/update?
5	Exactly when will it STOP - and which branch will run?

Method in Motion - Loop the Flow Master